

Clase 004 — Montaje del laboratorio: virtualización, Kali, snapshots y aislamiento de red

Parte: **0 — Fundamentos y prerequisites** · Fuente: *Kali Linux Documentation / OffSec* 🕒 Duración estimada: **120 min** · Nivel: **Fundamentos**

Objetivo

Construir un laboratorio de seguridad **aislado y reversible** en tu propio equipo. Practicar técnicas ofensivas requiere un entorno donde el error no tenga consecuencias fuera de él: una red que no toque Internet ni tu red doméstica, y máquinas que puedas devolver a un estado limpio en segundos. Al terminar tendrás una máquina atacante (Kali) y una o más máquinas víctima en una red interna sin salida, con snapshots que te permitan experimentar con malware o exploits sabiendo que cualquier daño es reversible con un clic.

⚠️ **Nota ética y de seguridad:** todo lo que se practica en este programa se hace **exclusivamente** dentro de este laboratorio aislado o contra sistemas para los que tengas autorización escrita. Atacar redes o equipos ajenos es ilegal. El aislamiento no es opcional: protege a terceros y te protege a ti.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Instalar** un hipervisor y comprobar la virtualización por hardware.
2. **Desplegar** Kali Linux y una VM víctima desde imágenes oficiales verificadas.
3. **Configurar** una red interna/host-only sin acceso a Internet.
4. **Gestionar** snapshots para revertir el estado tras cada práctica.
5. **Verificar** el aislamiento con pruebas de conectividad objetivas.
6. **Justificar** las decisiones de arquitectura del laboratorio (modo de red, recursos, higiene).

#	Tema	Por qué importa
1	Virtualización tipo 1 vs. tipo 2	Elegir hipervisor adecuado al equipo
2	VT-x/AMD-V	Sin ella, las VMs van lentísimas o no arrancan
3	Imágenes oficiales y verificación	Evitar ISOs manipuladas (checksum/firma)
4	Modos de red	NAT, bridged, host-only, internal
5	Aislamiento	Impedir fugas del laboratorio a producción
6	Snapshots	Reversibilidad = experimentar sin miedo
7	Máquinas víctima	Metasploitable, DVWA, VulnHub
8	Higiene del lab	Recursos, plantillas, mantenimiento

Explicación en profundidad

Virtualización: por qué el laboratorio vive en máquinas virtuales

Un hipervisor es el software que crea y ejecuta máquinas virtuales, entornos de cómputo completos aislados unos de otros y del anfitrión. Se distinguen dos tipos. El **tipo 1** (o *bare-metal*: ESXi, Hyper-V, Proxmox) corre directamente sobre el hardware, sin un sistema operativo anfitrión debajo, y ofrece el mejor rendimiento; es lo habitual en centros de datos. El **tipo 2** (VirtualBox, VMware Workstation) corre como una aplicación sobre tu sistema operativo normal, y es lo adecuado para un laboratorio personal porque es sencillo de instalar y convive con tu escritorio. Para que las VMs rindan de forma aceptable, el procesador debe ofrecer **virtualización asistida por hardware** (VT-x en Intel, AMD-V en AMD): sin ella, el hipervisor tiene que emular la CPU por software y las máquinas van desesperadamente lentas o directamente no arrancan sistemas de 64 bits. Esta extensión suele venir desactivada en la UEFI/BIOS y hay que habilitarla explícitamente.

La cadena de confianza: verificar la imagen antes de ejecutarla

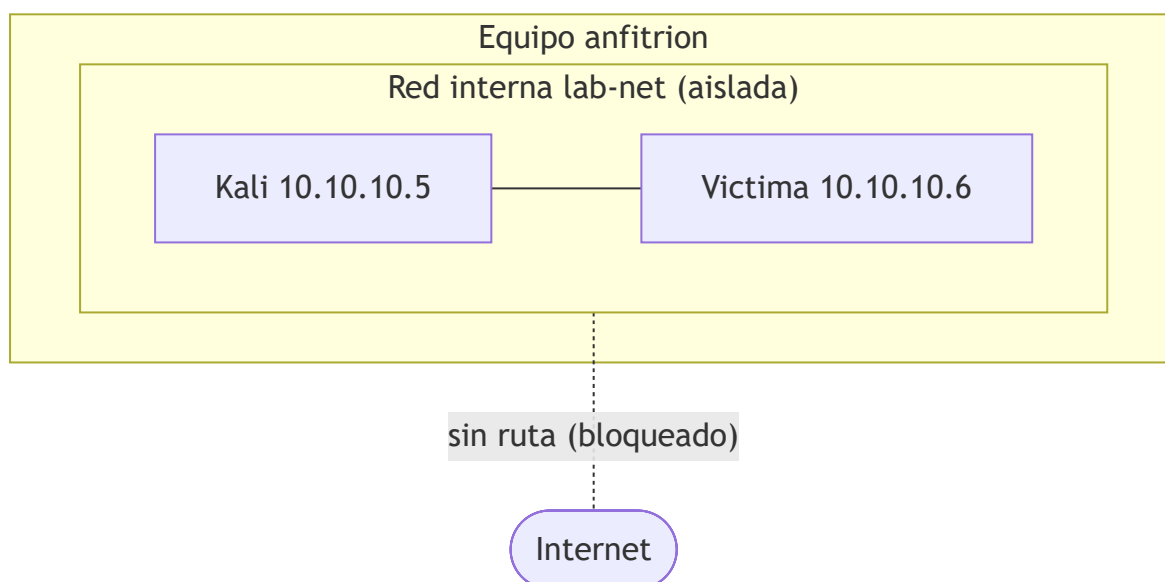
Descargar una ISO de un sistema de seguridad e instalarla sin verificarla es una contradicción: estarías confiando ciegamente en que nadie manipuló el archivo por el camino. Por eso los proyectos serios publican, junto a cada imagen, un **checksum** criptográfico (SHA-256) y, cuando se puede, una **firma** con su clave. El checksum te dice si el archivo que bajaste es idéntico bit a bit al que publicó el proyecto: recalculas el hash de tu descarga y lo comparas con el valor oficial. Si difiere, la descarga está corrupta o manipulada y **no** debes usarla. La firma va un paso más allá y demuestra que ese checksum lo publicó realmente el proyecto y no un impostor. Este hábito —verificar antes de ejecutar— es un reflejo profesional que aplicarás a cada binario que descargues en tu carrera.

Modos de red: el corazón del aislamiento

La decisión más importante del laboratorio es cómo conectas las VMs, porque de ahí depende que el entorno sea seguro. Los hipervisores ofrecen cuatro modos y cada uno tiene un nivel de aislamiento distinto. Elegir mal aquí es el fallo que convierte un laboratorio en un riesgo real.

Modo de red	Quién ve a quién	Salida a Internet	Uso típico
NAT	La VM sale, nadie entra a ella	Sí	Actualizar una VM puntualmente
Bridged (puente)	La VM es un equipo más de tu LAN	Sí	Casi nunca en un lab de seguridad
Host-only	VM ↔ host, pero no a Internet	No	Labs donde el host debe interactuar
Internal (interna)	Solo VM ↔ VM	No	Máximo aislamiento para malware/exploits

Para prácticas ofensivas la elección por defecto es **red interna**: las máquinas solo se ven entre ellas, ni siquiera el anfitrión tiene acceso directo, y no existe ruta hacia Internet ni hacia tu red doméstica. Esto es crítico cuando manejas máquinas deliberadamente vulnerables como Metasploitable, que jamás deben quedar expuestas, o cuando pruebas malware que intentaría "llamar a casa".



Snapshots: experimentar sin miedo

Un **snapshot** es una fotografía del estado completo de una VM en un instante: su disco y, opcionalmente, su memoria RAM. Es la característica que transforma el aprendizaje ofensivo, porque puedes ejecutar un exploit destructivo, infectar la máquina con malware o romper la configuración a propósito y luego **restaurar** el snapshot para volver en segundos a un punto limpio, como si nada hubiera pasado. La disciplina profesional es tomar un snapshot **base=limpia** de cada VM recién instalada y actualizada, y crear snapshots intermedios antes de cada experimento arriesgado. Un snapshot que incluye la RAM captura también el estado de ejecución (procesos abiertos), mientras que uno sin RAM equivale a apagar y volver a un disco anterior; el primero restaura más fielmente pero ocupa más. Ojo con acumular snapshots indefinidamente: cada uno consume espacio y, encadenados, degradan el rendimiento, así que conviene consolidar o eliminar los antiguos.

El flujo completo del montaje

Poniéndolo todo junto, el orden lógico de construcción es: comprobar la virtualización por hardware, instalar el hipervisor, verificar y desplegar Kali, verificar y desplegar la víctima, crear la red interna y conectar ambas, direccionar con IPs estáticas en la misma subred, probar que se ven entre sí, comprobar que **ninguna** alcanza Internet, y finalmente sellar el estado limpio con snapshots. Cada paso tiene una verificación objetiva —un `ping` que debe funcionar y otro que debe fallar— para que el aislamiento no sea una suposición, sino un hecho comprobado.

Definiciones y características

- **Hipervisor:** software que crea y ejecuta máquinas virtuales. El tipo 1 corre sobre el hardware (ESXi, Hyper-V); el tipo 2 sobre un sistema anfitrión (VirtualBox, VMware Workstation). Para un laboratorio personal, el tipo 2 es suficiente y más cómodo.
- **Virtualización asistida por hardware (VT-x/AMD-V):** extensiones del procesador que permiten ejecutar VMs con rendimiento aceptable. Sin ellas, la emulación por software hace las máquinas inutilizablemente lentas; suelen venir desactivadas en la UEFI.
- **Snapshot:** fotografía del estado completo de una VM (disco y opcionalmente RAM). Permite volver a un punto limpio en segundos, lo que hace seguro experimentar con exploits o malware.
- **Red host-only:** red virtual entre el anfitrión y las VMs, sin salida a Internet. Aísla del exterior pero el host sigue formando parte de la red, útil cuando el anfitrión debe interactuar con el lab.
- **Red interna (internal):** red que conecta solo a las VMs entre sí, sin que el host ni Internet tengan acceso. Es el nivel máximo de aislamiento y la opción por defecto para prácticas ofensivas.
- **NAT y bridged:** modos con salida a Internet. NAT permite a la VM salir sin ser alcanzable desde fuera; bridged la convierte en un equipo más de tu LAN. Ambos rompen el aislamiento y se evitan en un lab de seguridad salvo para una actualización puntual.
- **Checksum (SHA-256):** hash criptográfico que verifica que una descarga es idéntica bit a bit a la publicada. Si no coincide, la imagen está corrupta o manipulada y no debe usarse.
- **Firma:** verificación de que el checksum lo publicó realmente el proyecto y no un impostor, cerrando la cadena de confianza de la descarga.
- **Metasploitable:** máquina virtual deliberadamente vulnerable para practicar de forma legal. Nunca debe exponerse a Internet ni a redes con salida.
- **DVWA (Damn Vulnerable Web Application):** aplicación web insegura por diseño para practicar vulnerabilidades web dentro del laboratorio.

Glosario

Término	Definición concisa
Hipervisor	Software que ejecuta máquinas virtuales
Tipo 1 / bare-metal	Hipervisor que corre directo sobre el hardware
Tipo 2 / hosted	Hipervisor que corre sobre un sistema operativo anfitrión
VT-x / AMD-V	Virtualización asistida por hardware de Intel / AMD
VM	Máquina virtual: entorno de cómputo aislado
Snapshot	Fotografía restaurable del estado de una VM
Kali Linux	Distribución con herramientas de seguridad ofensiva
Metasploitable	VM deliberadamente vulnerable para practicar
DVWA	Aplicación web vulnerable por diseño
NAT	Modo de red: la VM sale, no es alcanzable desde fuera
Bridged	Modo de red: la VM es un equipo más de la LAN
Host-only	Red VM ↔ host, sin Internet
Internal	Red solo VM ↔ VM, aislamiento máximo
Checksum / SHA-256	Hash para verificar integridad de una descarga
OVA / OVF	Formato de empaquetado e importación de VMs

Herramientas y preparación

Instala **VirtualBox** (gratuito y multiplataforma) o **VMware Workstation Player**. Descarga la imagen oficial de **Kali Linux** desde <https://www.kali.org/get-kali/> y una máquina víctima como **Metasploitable 2** o **DVWA**. Verifica siempre el checksum SHA-256 publicado antes de usar cualquier imagen. Comprueba que la virtualización esté activada en la BIOS/UEFI. Como referencia de recursos, con 8 GB de RAM puedes correr Kali más una víctima con holgura; con 16 GB trabajarás cómodo con varios nodos.

Laboratorio guiado

1. **Comprobar la virtualización.** En Windows, abre el Administrador de tareas → Rendimiento → CPU y confirma "Virtualización: habilitada". Si no lo está, actívala en la UEFI (VT-x/AMD-V).
2. **Verificar la ISO de Kali.** Descarga la imagen y su checksum oficial. En PowerShell:

```
powershell Get-FileHash .\kali-linux-*.iso -Algorithm SHA256
```

Compara el resultado con el valor publicado en kali.org. Si no coincide, no la uses y vuelve a descargarla. 3. **Crear la VM Kali** en VirtualBox: 2 vCPU, 4 GB de RAM y 40 GB de disco. Monta la ISO y completa la instalación gráfica. 4. **Crear la red interna.** En VirtualBox, en la configuración de cada VM → Red, selecciona "Red interna" y ponle el nombre `lab-net`. Asigna ese adaptador tanto a Kali como a la víctima. 5. **Desplegar la víctima** (Metasploitable) importando su OVA/OVF y conectándola a `lab-net`. 6. **Direccionar.**

Configura IPs estáticas en la misma subred, por ejemplo Kali `10.10.10.5` y víctima `10.10.10.6`, con máscara `/24`. 7. **Probar la conectividad interna.** Desde Kali:

```
bash ping -c 3 10.10.10.6
```

Debe responder: las máquinas se ven entre sí. 8. **Verificar el aislamiento.** Desde Kali intenta salir a Internet; **no** debe existir ruta:

```
bash ping -c 2 8.8.8.8
```

Debe fallar (destino inalcanzable). Si responde, el adaptador no está en red interna. 9. **Tomar un snapshot** de cada VM en estado limpio recién instalado y actualizado. Etiqueta: `base-limpia`. 10. **Prueba de reversión.** Crea un archivo en Kali, apágala, restaura el snapshot `base-limpia` y confirma que el archivo desapareció: la reversibilidad funciona.

 **Nota ética:** la máquina víctima es vulnerable a propósito. No la conectes nunca a una red con salida ni a tu LAN doméstica; su único hogar es la red interna aislada.

Ejercicios

1. Explica cuándo usar NAT, bridged, host-only e internal, y por qué el laboratorio ofensivo usa internal por defecto.
2. Documenta el proceso completo de verificación de checksum y qué harías, paso a paso, si el hash no coincide.
3. Crea un tercer nodo (una VM Windows de evaluación) en la misma red interna y verifica que ve a Kali pero no a Internet.
4. Diseña una convención de nombres y de IPs para tu laboratorio, con subredes distintas por escenario.
5. Toma un snapshot con RAM y otro sin RAM sobre la misma VM y explica la diferencia práctica al restaurarlos.
6. Escribe un checklist de "higiene" para mantener el lab: actualizaciones, plantillas base, limpieza de snapshots y control de recursos.
7. Argumenta por qué verificar la firma, además del checksum, aporta una garantía que el checksum solo no da.

Reto verificable

Entrega un laboratorio funcional con al menos dos VMs (Kali más una víctima) en una red interna aislada, con IPs estáticas documentadas y un snapshot `base-limpia` por VM. Adjunta capturas de: (a) `ping` interno exitoso, (b) `ping` a Internet fallido, y (c) la lista de snapshots.

Criterio de aceptación: Kali alcanza a la víctima por la red interna pero **ninguna** VM alcanza Internet, y restaurar el snapshot devuelve la VM a un estado limpio verificable. Cualquier persona con tus notas debe poder reproducir el montaje desde cero.

⚠ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"VT-x is not available" o VM muy lenta	Virtualización desactivada en la UEFI o Hyper-V acaparando VT-x. Actívala; en Windows, desactiva Hyper-V si usas VirtualBox.
Las VMs no se ven entre sí	Adaptadores en redes distintas o IPs en subredes diferentes. Ponlas en el mismo <code>internal</code> y en la misma subred.
La víctima tiene salida a Internet	Adaptador en NAT o bridged por error. Cámbialo a red interna y vuelve a comprobar con <code>ping 8.8.8.8</code> .
Snapshot enorme o disco lleno	Snapshots acumulados sin limpiar. Consolida o elimina los antiguos que ya no necesitas.
La ISO no arranca	Descarga corrupta. Verifica el checksum y vuelve a bajarla desde la fuente oficial.
El <code>ping</code> interno falla pese a estar en <code>lab-net</code>	Firewall de la VM bloqueando ICMP o IPs en subredes distintas. Revisa el firewall y el direccionamiento.

? Preguntas frecuentes

? **¿Puedo usar Kali como sistema principal?** No es recomendable para empezar: Kali está diseñado como caja de herramientas, no como escritorio diario, y correrlo como anfitrión te expone innecesariamente. Úsalo dentro de una VM aislada.

? **¿VirtualBox o VMware?** Ambos sirven para el curso. VirtualBox es gratuito y multiplataforma; VMware suele rendir algo mejor. Elige uno y sé consistente para no dispersarte con dos flujos distintos.

? **¿Por qué red interna y no host-only?** Host-only deja al anfitrión dentro de la red del lab; la red interna aísla aún más, dejando las VMs solas entre sí. Para prácticas con malware, cuanto más aislado, mejor y menos riesgo para el host.

? **¿Necesito mucha RAM?** Con 8 GB puedes correr Kali más una víctima. Con 16 GB trabajas cómodo con varios nodos simultáneos. La RAM suele ser el recurso que primero se agota al añadir máquinas.

? **¿Cada cuánto tomo snapshots?** Uno `base-limpia` tras instalar y actualizar cada VM, y uno más antes de cualquier experimento arriesgado. Después, limpia los que ya no aporten para no llenar el disco ni degradar el rendimiento.

🔗 Referencias

- Kali Linux — Get Kali y documentación — <https://www.kali.org/docs/>
- Oracle VirtualBox Manual — <https://www.virtualbox.org/manual/>
- Rapid7 Metasploitable 2 — <https://docs.rapid7.com/metasploit/metasploitable-2/>
- OWASP DVWA — <https://github.com/digininja/DVWA>

📄 Material descargable

- 📄 [Guía en PDF](#) — versión imprimible de esta clase.

-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

Clase anterior

[Clase 003](#) — Frameworks de seguridad: NIST CSF, ISO 27001, MITRE ATT&CK y Diamond Model

Siguiente clase

[Clase 005](#) — Linux esencial para seguridad: filesystem, permisos y usuarios